

数据库应用开发：社交媒体活动的情感倾向监测与时空分布分析

数据库系统原理课程设计报告



学 号 _____

姓 名 _____

专 业 _____

授课老师 _____

目录

一. 概述.....	4
1.1 课题背景.....	4
1.2 编写目的.....	4
二. 需求分析.....	4
2.1 功能需求.....	4
2.2 数据字典.....	5
2.3 数据流图.....	6
三. 可行性分析.....	7
3.1 技术可行性.....	7
3.2 应用可行性.....	7
四. 概念设计.....	8
4.1 实体.....	8
4.2 实体属性局部 E-R 图.....	9
4.3 全局 E-R 图.....	11
五. 逻辑设计.....	13
5.1 E-R 图向关系模型的转变.....	13
5.2 数据模型的优化及规范化设计.....	15
5.2.1 第三范式（3NF）验证.....	15
5.2.2 索引设计与优化.....	16
六. 项目管理.....	16
6.1 框架选择.....	16
6.2 开发平台.....	17
七. 系统实现.....	18
7.1 系统架构搭建.....	18
7.2 系统逻辑设计.....	19
7.2.1 总体逻辑流程描述.....	19
7.2.2 功能模块逻辑设计.....	19
7.2.3 底层功能模块协作逻辑.....	21
7.2.4 异常处理与安全逻辑.....	22
7.3 具体功能编写.....	22
7.3.1 数据库建表与初始化.....	22
7.3.2 逻辑模块编写.....	25
7.3.3 底层功能模块编写.....	26
7.4 功能测试.....	26
7.4.1 用户认证测试.....	26
7.4.2 用户管理测试.....	27
7.4.3 帖子数据管理测试.....	28
7.4.4 帖子查询与可视化测试.....	31
八. 总结.....	34
参考文献.....	35

摘要 随着社交媒体平台用户数量的急剧增加，海量且实时的文本、多媒体及位置信息不断涌现。如何从这些异构数据中抽取时空特征与情感倾向，并加以存储与分析，成为政府部门、企业和研究机构进行舆情监测与决策支持的关键需求。本项目基于关系型数据库技术，设计并实现了一个“社交媒体活动的情感倾向监测与时空分布”系统，主要功能包括社交媒体数据预处理、情感分析与标签提取、时空分布存储与多维查询、可视化报表与接口服务。通过该系统，用户能够实时监控城市范围内不同区域的社交媒体活跃情况与公众情感变化，为突发事件预警、舆情分析与市场调研提供数据支撑。在高并发与海量数据场景下，得益于合理的表设计、索引优化与分区策略，系统依然保持良好的查询性能与可扩展性。该项目可进一步扩展到大数据平台环境下，结合分布式存储与流式计算，提升时效性与处理规模，以满足更大范围、多源异构社交数据的实时分析需求。

关键词：数据库；社交媒体；情感分析

一. 概述

1.1 课题背景

随着社交媒体的普及，用户通过在社交媒体上发布帖子、评论、图片等活动创造了大量的数据，这些数据构成了丰富的情感信息来源。通过对这些数据进行时空分布和情感倾向的分析，可以及时捕捉热点话题、评估公众情绪，辅助政府或企业采取应对策略、及时响应各类事件。

基于此需求，设计并实现一个“社交媒体活动的时空分布与情感倾向监测”数据库应用系统，通过实时采集、存储、分析多平台社交媒体数据，并提供多维度查询与可视化服务，从而实现对城市范围内舆情和情感态度的监控与预警。

1.2 编写目的

本数据库系统的设计旨在借助数据采集与情感分析等技术手段，动态掌握城市不同区域内公众情绪的变化趋势，实现情感监测。其核心目标是为突发事件预警、社会舆情解读以及市场调研等应用场景提供有力的数据支持。同时，系统具备向大数据平台拓展的潜力，可融合分布式存储与流式处理技术，进一步提升数据处理的效率与规模，以应对更广泛范围及多源异构社交媒体数据的实时分析需求。

二. 需求分析

2.1 功能需求

结合项目目标，系统的主要功能需求包括以下四个方面：

（1）数据采集与预处理需求。需要支持从不同数据源（社交平台等）采集原始文本、图片、视频等多种格式的数据；支持从历史数据批量导入，同时可扩展至实时抓取动态生成的活动数据；支持对数据进行简单的预处理，包括统一采集数据的格式、去除噪音数据、分词等工作。

（2）数据分析需求。需要集成情感分析模型，支持对文本数据进行正负面分类、计算多维度的情感得分，并提取文本的话题标签；结合情感得分以及话题、关键字、时间地点等信息，在时空维度上对情感进行趋势分析和热点识别。

（3）数据统计与查询需求。需要支持基于用户、时间、空间、关键词和情感类别的多维度联合查询；支持数据可视化，能够基于情感分布、话题热度、时空分布生成数据图表；支持数据导出，可按需求导出原始数据和分析后数据的统计报表。

（4）用户管理需求。需要提供基础的用户管理功能，包括对系统用户的创建、查询、

删除及密码修改，并支持账号激活/冻结等状态控制；采用基于角色的访问控制，为用户分配管理员、数据分析师、查询用户等不同权限角色，保证数据导入、查询与管理操作在安全范围内进行。

2.2 数据字典

该数据库的数据字典由字段名、数据类型、约束控制及说明组成。具体的数据字典设计如表 2.1 所示。

表 2.1 数据字典设计

字段名	类型	约束	说明
post_id	INT	主键，非空自增	社交媒体内容（帖子）id
user_id	INT	非空，外键	帖子发布者的用户 id
content	TEXT	可为空	帖子文本内容
location	VARCHAR(255)	可为空	帖子发布地理位置
create_time	DATETIME	非空	帖子创建时间
content_hash	CHAR(32)	可为空	帖子文本内容的 md5 hash 结果
user_id	INT	主键，非空自增	用户 id
username	VARCHAR(100)	非空	用户名
platform	VARCHAR(100)	非空	用户所在的社交媒体平台
gender	VARCHAR(40)	非空，只能为 Female、Male、Unknown	用户性别
birthday	DATE	可为空	用户出生年月日
sentiment_id	INT	主键，非空自增	情感分析 id
post_id	INT	非空，外键	情感分析对应的帖子 id
score	DECIMAL(5,4)	非空	情感分析得分
type	VARCHAR(40)	非空	情感类型
model	VARCHAR(100)	非空	使用的情感分析模型
analysis_time	DATETIME	非空	进行情感分析的时间
topic_id	INT	主键，非空自增	话题 id
topic	VARCHAR(100)	非空，唯一	话题标签
description	VARCHAR(255)	可为空	话题描述
keyword_id	INT	主键，非空自增	关键词 id
keyword	VARCHAR(100)	非空，唯一	关键词
post_id	INT	联合主键，非空，外键	帖子-话题关系中的帖子 id
topic_id	INT	联合主键，非空，外键	帖子-话题关系中的话题 id
post_id	INT	联合主键，非空，外键	帖子-关键词关系中的帖子 id
keyword_id	INT	联合主键，非空，外键	帖子-关键词关系中的关键词 id
user_id	INT	主键，非空自增	数据库系统的用户 id

username	VARCHAR(150)	非空，唯一	系统用户名
password_hash	VARCHAR(255)	非空	系统用户的密码加密串
is_active	TINYINT(1)	非空，默认为 1	系统用户激活状态
created_at	DATETIME	非空，默认为当前时间	系统用户创建时间
role_id	SMALLINT	主键，非空自增	数据库系统的角色 id
name	VARCHAR(50)	非空，唯一	系统角色名
description	VARCHAR(255)	可为空	系统角色描述
user_id	INT	联合主键，非空，外键	系统用户-角色关系中的用户 id
role_id	SMALLINT	联合主键，非空，外键	系统用户-角色关系中的角色 id

2.3 数据流图

(1) 数据采集和预处理子系统

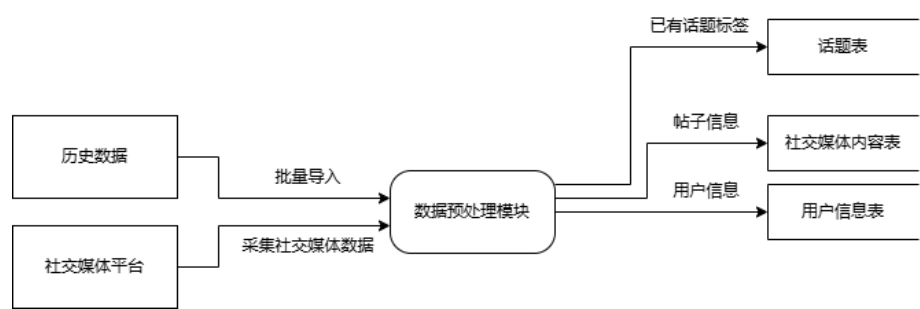


图 2.1 数据采集和预处理子系统数据流图

(2) 情感分析与标签提取子系统

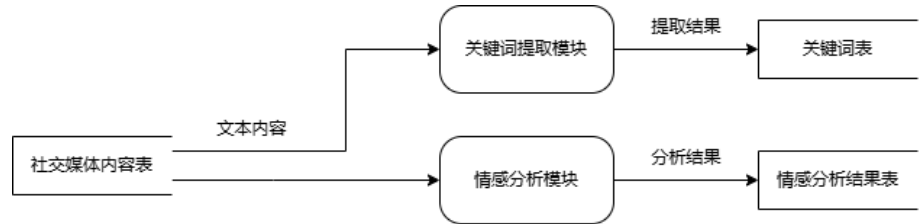


图 2.2 情感分析与标签提取子系统数据流图

(3) 时空分布统计与查询子系统

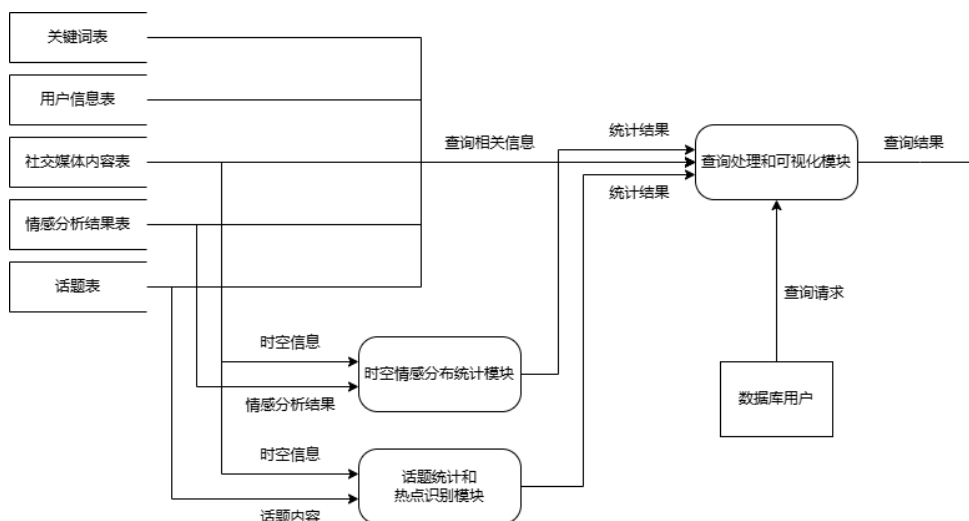


图 2.3 时空分布统计与查询子系统数据流图

三. 可行性分析

3.1 技术可行性

基于下列提到的开源技术与成熟平台，能够满足该数据库系统开发的各项技术需求：

（1）数据库技术方面，选用 MySQL 8.0 和 InnoDB 存储引擎，支持 ACID 事务、外键约束，并具备分区、行锁等特性，满足高并发写入和查询的需求。

（2）情感分析模型方面，可使用 BERT、RoBERTa 等预训练模型微调后进行中文情感分类，或者利用已有的开源 API 服务，实现单条文本情感打分。

（3）关键词与话题提取方面，可利用基于统计的 TF-IDF、TextRank 等算法完成关键词抽取与话题聚类，也可以借助 KeyBERT 等 NLP 模型完成处理。

（4）后端开发环境方面，可使用基于 Java 的 Spring Boot 框架或基于 Python 的 Flask 框架等，构建数据采集、预处理与情感分析等服务；前后端分离模式下，前端可使用 Vue.js 或 React.js 进行可视化展示。

3.2 应用可行性

该数据库系统在现实层面具有应用价值，主要体现在以下几个方面：

（1）满足社会需求。随着互联网舆情的重要性日益凸显，政府部门、企业与研究机构对实时监测社交媒体数据的需求持续增长，尤其在突发事件、公共安全和市场营销等领域，时空情感监测具有显著价值。

（2）数据可获得性拥有保障。各个社交平台实时生产大量的社交媒体资源，且多数提供开放的 API 或可通过爬虫工具合法获取公开内容；同时，内容中往往包含发布时间和地

理位置信息，为时空分析提供数据基础。

(3) 拥有一定的商业价值。基于情感趋势与热点话题统计，可以为舆情预警、市场调研、公共政策制定等提供决策支持；时空情感监测系统还可延伸为商业情报平台、品牌舆情监控等增值服务。

四. 概念设计

4.1 实体

该数据库的概念模型由七类主要实体及其相互关系构成，旨在刻画社交媒体数据的内容、话题标注、关键词提取与情感分析过程，同时体现基于用户管理的数据库安全性设计。

实体集包括以下七个模块：

(1) **users** (用户管理模块)：管理系统记录的所有发布内容的用户信息。记录用户的用户名、性别、年龄等基本信息，统计用户分布、辅助建立用户行为画像。

(2) **posts** (帖子管理模块)：管理用户在社交平台上发布的单条消息、动态等。记录每条动态的基本信息，包括发布者、文本内容、发布位置和时间等，其内容用于实现情感分析、分析特定用户的行为模式、统计城市中社交活动和情感趋势的时空分布变化等。

(3) **topics** (话题管理模块)：管理人为或自动预定义的一组主题标签，用于对帖子进行宏观分类。标签可以直接从各社交媒体平台上提取，也可以从帖子的原始内容中分析得出。

(4) **keywords** (关键词管理模块)：管理从帖子文本中自动抽取的高频或重要词汇。具体的关键词由后期算法分析生成。

(5) **sentiments** (情感分析结果管理模块)：管理对每条帖子内容进行自然语言处理后得出的情感倾向及相关元信息。

(6) **sys_users** (系统用户管理模块)：管理数据库应用系统的用户信息。记录用户名、密码、激活状态、创建时间等信息。

(7) **sys_roles** (系统角色管理模块)：管理数据库应用系统的角色信息。每个角色在系统中对应不同的访问和操作权限，一个系统用户可以拥有多个角色。

各个实体集之间通过关系连接。具体来说，**posts** 模块和其他各模块之间分别建立一个关系，其中和 **users**、**sentiment** 之间关系的映射基数为一对多，和 **topics**、**keywords** 之间关系的映射基数为多对多；**sys_users** 和 **sys_roles** 之间建立多对多关系。

为实现多对多关系，还设计了以下三个弱实体：

(1) **tag** (帖子 - 话题关系)：描述帖子包含话题的关系。映射每个帖子与其所属的一个或多个话题标签。

(2) **extract** (帖子 - 关键词关系)：描述帖子包含关键词的关系。映射每个帖子与其包含的一个或多个关键词。

(3) **assign** (系统用户 - 系统角色关系)：描述系统用户拥有角色的关系。映射每个用户与其拥有的一个或多个用户。

4.2 实体属性局部 E-R 图

(1) users (用户管理模块)

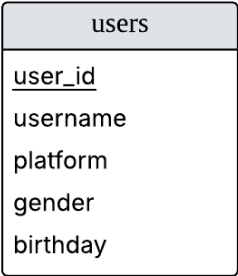


图 4.1 users 模块局部 E-R 图

(2) posts (帖子管理模块)

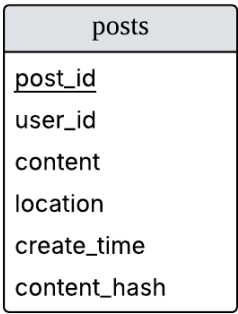


图 4.2 posts 模块局部 E-R 图

(3) topics (话题管理模块)

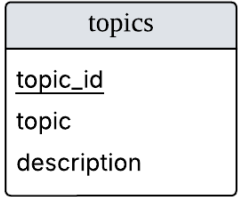


图 4.3 topics 模块局部 E-R 图

(4) keywords (关键词管理模块)

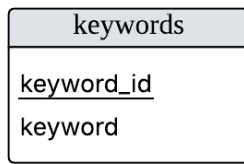


图 4.4 keywords 模块局部 E-R 图

(5) sentiments (情感分析结果管理模块)

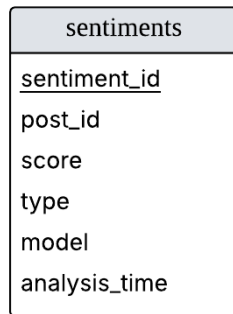


图 2.5 sentiments 模块局部 E-R 图

(6) sys_users (系统用户管理模块)

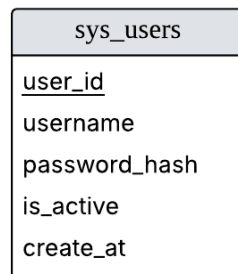


图 2.6 sys_users 模块局部 E-R 图

(7) sys_roles (系统角色管理模块)

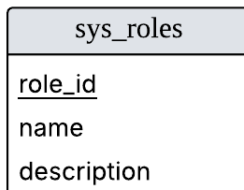


图 2.7 sys_roles 模块局部 E-R 图

(8) tag (帖子 - 话题关系)

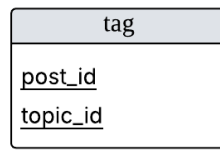


图 4.8 tag 关系局部 E-R 图

(9) extract (帖子 - 关键词关系)

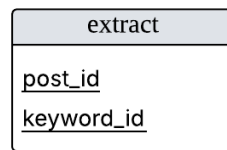


图 4.9 extract 关系局部 E-R 图

(10) assign (系统用户 - 系统角色关系)

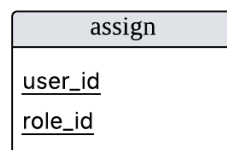


图 4.10 assign 关系局部 E-R 图

4.3 全局 E-R 图

对每个实体集和关系进行详细设计后，绘制数据库整体的 E-R 图。

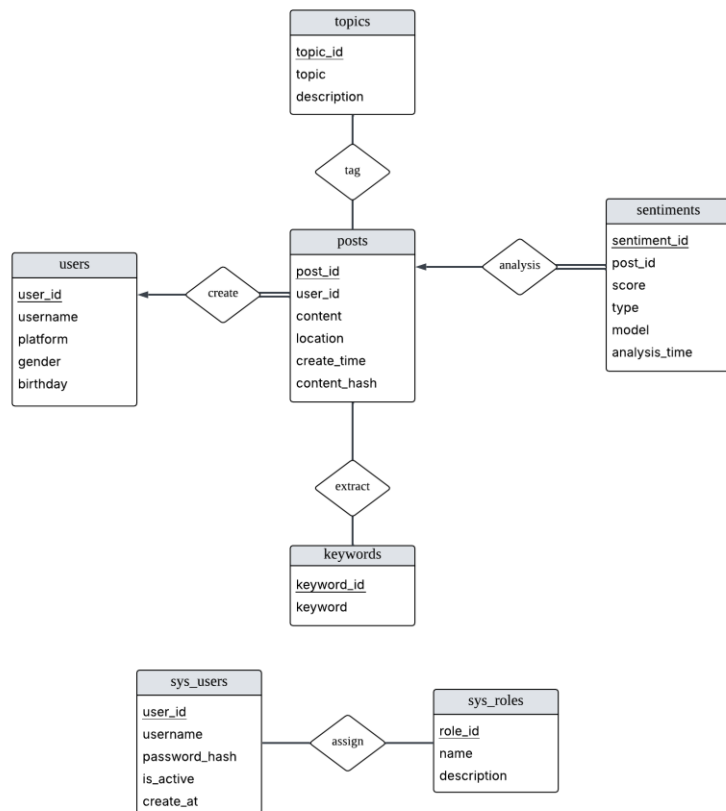


图 4.9 数据库整体 E-R 图

五. 逻辑设计

5.1 E-R 图向关系模型的转变

将以上概念设计中各实体及关系，转换为以下关系模型：

（1）社交媒体内容信息表 posts

表 5.1 社交媒体内容信息表 posts

表名：posts			
字段	类型	约束	说明
post_id	INT	主键，非空自增	社交媒体内容 id
user_id	INT	非空，外键，关联 users 表的 user_id	用户 id
content	TEXT	可为空	文本内容
location	VARCHAR(255)	可为空	发布位置
create_time	DATETIME	非空	创建时间
content_hash	VARCHAR(32)	可为空	文本 md5 hash 结果
联合唯一键 (user_id, content_hash, create_time)			

（2）用户信息表 users

表 5.2 用户信息表 users

表名：users			
字段	类型	约束	说明
user_id	INT	主键，非空自增	用户 id
username	VARCHAR(100)	非空	用户名
platform	VARCHAR(100)	非空	用户所在的社交媒体平台
gender	VARCHAR(40)	非空，只能为 Female、Male、Unknown	性别
birthday	DATE	可为空	出生年月日
联合唯一键 (username, platform)			

（3）情感分析结果信息表 sentiments

表 5.3 情感分析结果信息表 sentiments

表名：sentiment			
字段	类型	约束	说明
sentiment_id	INT	主键，非空自增	情感分析 id
post_id	INT	非空，外键，关联 posts 表的 post_id	社交媒体内容 id

score	DECIMAL(5,4)	非空	情感分析得分
type	VARCHAR(40)	非空	情感类型
model	VARCHAR(100)	非空	使用的情感分析模型
analysis_time	DATETIME	非空	进行情感分析的时间
联合唯一键 (post_id, model)			

(4) 话题信息表 topics

表 5.4 话题信息表 topics

表名: topics			
字段	类型	约束	说明
topic_id	INT	主键, 非空自增	话题 id
topic	VARCHAR(100)	非空, 唯一	话题标签
description	VARCHAR(255)	可为空	话题描述

(5) 关键词信息表 keywords

表 5.5 关键词信息表 keywords

表名: keywords			
字段	类型	约束	说明
keyword_id	INT	主键, 非空自增	关键词 id
keyword	VARCHAR(100)	非空, 唯一	关键词

(6) 帖子-话题关联表 posts_topics

表 5.6 帖子-话题关联表 posts_topics

表名: posts_topics			
字段	类型	约束	说明
post_id	INT	联合主键, 非空, 外键, 关联 posts 表的 post_id	社交媒体内容 id
topic_id	INT	联合主键, 非空, 外键, 关联 topics 表的 topic_id	话题 id

(7) 帖子-关键词关联表 posts_keywords

表 5.7 帖子 - 关键词关联表 posts_keywords

表名: posts_keywords			
字段	类型	约束	说明
post_id	INT	联合主键, 非空, 外键, 关联 posts 表的 post_id	社交媒体内容 id
keyword_id	INT	联合主键, 非空, 外键, 关联 keywords 表的 keyword_id	关键词 id

(8) 系统用户信息表 sys_users

表 5.8 系统用户信息表 sys_users

表名: media			
字段	类型	约束	说明
user_id	INT	主键, 非空自增	用户 id
username	VARCHAR(150)	非空, 唯一	用户名
password_hash	VARCHAR(255)	非空	密码加密串
is_active	TINYINT(1)	非空, 默认为 1	用户激活状态
create_at	DATETIME	非空, 默认为当前时间	用户创建时间

(9) 系统角色信息表 sys_roles

表 5.9 系统角色信息表 sys_roles

表名: media			
字段	类型	约束	说明
role_id	INT	主键, 非空自增	角色 id
name	VARCHAR(150)	非空, 唯一	角色名
description	VARCHAR(255)	可为空	角色描述

(10) 系统用户-角色关联表 sys_users_roles

表 5.10 系统用户-角色关联表 sys_users_roles

表名: media			
字段	类型	约束	说明
user_id	INT	联合主键, 非空, 外键, 关联 sys_users 表的 user_id	用户 id
role_id	SMALLINT		角色 id

上述关系模型一一对应概念设计中的实体与关系，多对多映射通过关联表实现。

5.2 数据模型的优化及规范化设计

5.2.1 第三范式（3NF）验证

各主要关系均设计为将属性完全依赖于各自主键，避免传递或部分依赖，满足 3NF。具体依赖情况如下：

- (1) 对于 posts 表，content、latitude、longitude、create_time 完全依赖于主键 post_id，

user_id 为外键。

(2) 对于 users 表, username、platform、gender、birthday 均完全依赖于主键 user_id。

(3) 对于 sentiment 表, score、type、model、analysis_time 均完全依赖于主键 sentiment_id, post_id 为外键。

(4) 对于 topics 表, topic、description 均完全依赖于主键 topic_id。

(5) 对于 keywords 表, keyword 完全依赖于主键 keyword_id。

(6) 对于 sys_users 表, username、password_hash、is_activate、create_at 均完全依赖于主键 user_id。

(7) 对于 sys_roles 表, name、description 均完全依赖于主键 role_id。

(8) 对于 posts_topics 表、posts_keywords 表和 sys_users_roles 表, 均采用复合主键, 且所有属性为外键引用, 没有冗余属性。

5.2.2 索引设计与优化

针对各个表的具体查询需求, 除了各主键自动建立聚集索引外, 构建其他非聚集索引、提高查询效率。

(1) posts 表: 对外键 user_id 建立索引, 便于快速查找某用户的帖子; 对 create_time 和 location 分别建立索引, 加速时空查询。

(2) users 表: 对 username 建立索引, 支持根据用户名快速检索用户信息。

(3) sentiment 表: 对外键 post_id 建立索引, 加速对某帖子的情感数据和媒体资源的查询。

(4) topics 表、keywords 表: 对 topic 和 keyword 分别建立索引, 加速根据内容查找话题和关键词。

(5) sys_users 表: 对 username 建立索引, 加速通过用户名查询用户信息。

(6) sys_roles 表: 无额外索引。

(7) post_topic 表、post_keyword 表、sys_users_roles: 分别对 post_topics、post_keywords 两表的 topic_id 和 keyword_id 属性构建单列索引, 支持根据特定话题和关键词查询帖子信息。

另外, 使用 MySQL 内建的统计信息和优化器, 定期更新索引统计信息, 确保索引选择优化查询计划, 从而实现索引的自动维护; 监控索引使用情况, 删除冗余且使用频率低的索引, 对高频查询的字段调整高效的索引类型。

六. 项目管理

6.1 框架选择

本系统需实现社交媒体数据的批量导入、情感分析、复杂多条件查询、时空可视化与报

表导出，并支持基本的用户与权限管理。系统需在课程项目条件下做到：实现性强、易开发、可扩展、性能可接受、便于部署与演示。

为兼顾开发效率与可维护性，本项目的总体架构采用 B/S 模式，并采用三层分层架构：表示层负责 UI 渲染与用户交互，应用层负责处理业务逻辑，数据层负责数据存储；表示层与应用层分离，职责清晰。

系统的具体组件与技术栈选择如下。

（1）前端

基础框架采用 HTML + CSS + JavaScript。使用 Tailwind CSS，提供实用类快速构建 UI，适合课程项目；模块化构建 JavaScript，各页面拆分成独立 JS 文件，保持职责单一，同时便于页面间共享头部、导航栏与状态。

可视化功能方面，使用 Chart.js 绘制条形图 / 折线图 / 饼图，词云和热力图则通过显示后端生成的 PNG 和 folium HTML 实现。

（2）后端

基础框架采用 Python + Flask，轻量、灵活，便于自定义路由，配合 flask-jwt-extended 实现 JWT 认证、token 管理、错误回调。

数据库连接方面，直接使用 mysql-connector 驱动，以手写 SQL 为主、精细控制批量插入与性能。

可视化功能方面，分别通过 wordcloud 和 folium 库生成词云和热力图，生成结果以文件形式传输至前端显示。使用 geopy 将地理位置转化成热力图所需的经纬度信息，由于该 API 访问速度受限，故使用缓存提高效率。

情感分析方面，提供 TextBlob 和 BERT 两种模型选择。TextBlob 轻量、适用于快速实验性的情感分类，配合翻译 API 实现不同语言内容的情感分析；Transformers + BERT 更高精度但计算开销大，适用于批量、异步处理。

关键词提取方面，使用效率较高的 TfIdf 分词。引入分词库 jieba 和 janome 对中文和日文进行分词，从而支持中 / 日 / 英三种语言的关键词提取。

安全性相关方面，使用 bcrypt 进行密码哈希。

（3）数据库

使用事务支持、熟悉度高的 MySQL (InnoDB)，MySQL 常见于教学与小型生产环境都很，支持事务、索引、唯一约束与高效批量写入 (LOAD DATA)，且与 Python 驱动适配成熟。采用 content_hash + UNIQUE(user_id, content_hash, create_time) 等联合唯一键实现插入，对高频查询字段建立索引。Docker 化部署 MySQL，便于演示与复现。

6.2 开发平台

操作系统：Windows 11（64 位）

开发工具：Visual Studio Code

版本控制：Git 2.50.0

开发语言：Python 3.13.2

前端工具链：HTML + Tailwind CSS 4.1.14 + JavaScript

后端框架：Flask 3.1.2，包含 flask-jwt-extended、mysql-connector 等扩展

数据库版本：MySQL 9.4.0

依赖管理：使用 venv + pip 管理 Python 依赖，使用 npm 管理 JavaScript 依赖，使用 Docker 运行 MySQL

七. 系统实现

7.1 系统架构搭建

在 6.1 总体架构理念的基础上，详细设计系统架构的实际部署与搭建过程：

(1) 表示层（前端）：采用 HTML + CSS + 模块化原生 JavaScript，模块化文件包括 manage.js、query.js、user.js，负责页面渲染、用户交互与向后端调用 API。页面采用模板化思路，拆分为主框架模板 + 数据查询、数据管理、用户管理三个子页面模板，便于维护与复用。

(2) 应用层（后端）：使用 Python + Flask 提供 API，负责业务逻辑、认证鉴权、批量数据处理、情感分析调用、可视化数据聚合及文件导出。将 Flask 应用划分为四个功能模块，包括 auth（认证）、posts（导入 / 导出 / 查询 / 管理）、users（系统用户管理）、visual（可视化接口）。同时，后端还负责数据校验、事务控制、批量插入 / 更新逻辑、触发情感分析模型、生成部分可视化资源及 CSV 格式文件导出等底层逻辑。

(3) 数据层：MySQL（InnoDB），设计合理的表结构、联合唯一约束、索引与分区（如按时间分区 posts 表可选），支持高并发插入与复杂聚合查询。

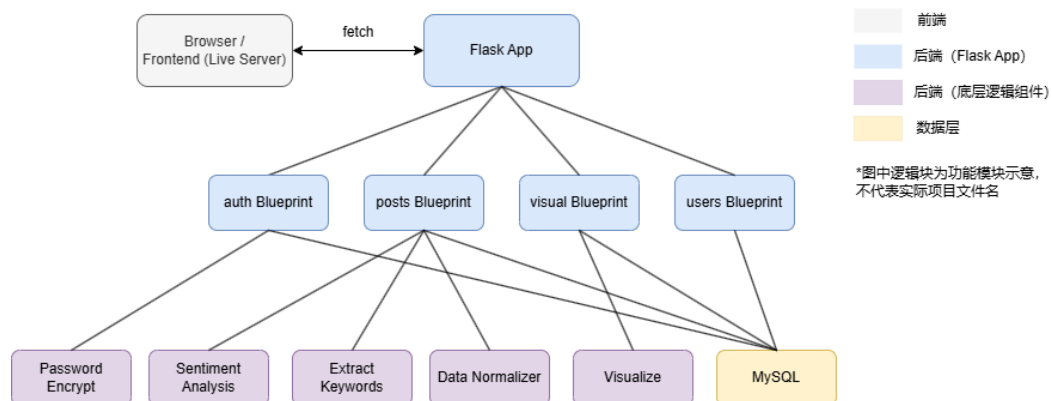


图 7.1 系统架构示意图

系统架构的搭建过程如下。

(1) 前端：使用 Live Server 启动前端页面，前端与后端通过 HTTPS 请求交互数据，端口 5500。

(2) 后端：使用 Python 虚拟环境安装相关依赖，创建 Flask 项目结构，包括 app.py

和蓝图目录；启动 Flask 应用程序，监听端口 5000，等待前端请求。

（3）数据库：使用 Docker 拉取 MySQL 镜像，启动 MySQL 容器并映射端口 3307:3306。在 Python 中配置数据库连接信息，端口为 3307，通过 mysql-connector 连接 MySQL，并初始化数据库和数据表内容。

7.2 系统逻辑设计

7.2.1 总体逻辑流程描述

系统采用前后端分离架构。前端页面通过 fetch() 调用后端 Flask 提供的 API，实现用户认证、数据查询、情感分析与可视化等功能；后端根据请求类型将任务分发至对应的功能模块（Blueprint），各功能模块再调用底层逻辑组件或数据库完成处理。

7.2.2 功能模块逻辑设计

在系统的三层架构中，将应用层 Flask 应用的业务逻辑细分为多个功能模块，每个模块实现一组特定领域的功能。主要模块的划分及 API 设计如下。

（1）auth（用户认证模块）
实现用户的注册、登录、修改密码、验证与 JWT token 签发等功能。

表 7.1 用户认证模块 API 设计

API	方法	功能	逻辑细节
/auth/login	POST	处理用户登录	从前端接受用户名和密码，结合数据表验证成功后生成并向前端返回 token
/auth/register	POST	处理用户注册	接受用户名和密码，数据表中插入相应用户
/auth/refresh	POST	刷新 token	检查 refresh_token 有效性，若有效则刷新已登录用户的 access_token
/auth/me	GET	获取用户基本信息	从 access_token 中获取用户名，从数据表中查询并返回该用户的基本信息，包括用户名和角色
/auth/modify_password	POST	处理用户修改密码	验证用户的旧密码，通过后将数据表中的密码修改为新密码

（2）posts（帖子数据管理模块）
实现社交媒体原始数据的批量导入、导出、条件查询、删除和管理情感分析记录等附属信息的功能。

表 7.2 帖子数据管理模块 API 设计

API	方法	功能	逻辑细节
/posts/import	POST	导入数据	从前端接受上传的文件，暂存、读取文件，调用接口处理数据并导入数据库
/posts/export	GET	导出数据	按条件查询数据，将查询结果转换为标准格式并写入文件，将结果文件传输给前端
/posts/query	GET	按条件查询数据	从前端获取查询条件（时间、内容、话题、关键词、平台、情感分析记录、情感类型），向数据库查询相关帖子记录并返回前端
/posts/delete	POST	删除数据	从前端获取帖子 id 列表，从数据库中删除相关帖子记录
/posts/delete_sentiments	POST	删除帖子的情感分析数据	从前端获取情感分析模型名称和帖子 id 列表，从数据库中删除相关的情感分析记录
/posts/reanalyze	POST	重新对帖子进行情感分析	从前端获取情感分析模型名称和帖子 id，调用接口、进行情感分析，并将结果插入数据库

（3）visual（可视化模块）

实现数据查询页面的可视化查询逻辑，包括统计情感分布、生成情感时间趋势、统计话题热度、生成词云和热力图等功能。

表 7.3 可视化模块 API 设计

API	方法	功能	逻辑细节
/visual/sentiment_distribution	GET	获取情感分布	按条件查询数据，从结果中统计各情感类型的帖子数、返回前端
/visual/sentiment_trend	GET	获取情感时间趋势	按条件查询数据，将查询结果按时间进行分组排序，再以日期为单位统计各情感类型的帖子数，结果返回前端
/visual/top_words	GET	获取词分布	按条件查询数据，从查询结果中统计词频、返回前端，词数量通过前端传递参数指定；话题词和关键词分布共用该 API，通过前端发送的 mode 参数决定查询对象
/visual/wordcloud	GET	生成关键词云	按条件查询数据，从查询结果中统计词频，调用接口生成 PNG 格式的词云文件、传输给前端； 话题词和关键词分布共用该 API，具体形式与 /visual/top_words 相同
/visual/heatmap	GET	生成情感地理热力图	按条件查询数据，从查询结果中提取情感分数及地理位置信息，调用接口生成 HTML 格式的热力图文件、传输给前端

(4) users (用户管理模块)

实现对用户的查询、删除、激活和冻结、重置密码、更改角色功能。

表 7.4 用户管理模块 API 设计

API	方法	功能	逻辑细节
/users/query	GET	查询用户	从前端获取查询条件（用户名），向数据库查询相关用户和角色列表并返回前端
/users/delete	POST	删除用户	从前端获取用户 id 列表，从数据库中删除相关用户
/users/batch_update	POST	用户批量更新 (激活 / 冻结 / 重置密码)	从前端获取操作类型和用户 id 列表，对数据库相关用户元组执行指定更新操作（激活 / 冻结 / 重置密码）； 激活、冻结、重置密码功能共用该 API，通过前端发送的 action 参数决定操作类型
/users/batch_set_role	POST	批量更改用户角色	从前端获取操作类型、角色名和用户 id 列表，在数据库中对相关用户执行指定角色的赋予或回收操作； 赋予角色、回收角色功能共用该 API，通过前端发送的 action 参数决定操作类型

7.2.3 底层功能模块协作逻辑

底层功能模块独立于 Flask 视图层，由一个或多个 Blueprint 调用。主要的底层功能模块设计如下。

- (1) Password Encrypt: 实现密码加密与校验逻辑。被 auth 模块调用。
- (2) Sentiment Analysis: 实现文本情感分类和打分，提供多个情感模型、对外提供批处理接口。被 posts 模块调用。
- (3) Extract Keywords: 实现关键词提取，对外提供批处理接口。被 posts 模块调用。
- (4) Data Normalizer: 实现数据清洗与格式化，提供多个接口、将不同平台采集的不同格式的数据转化成能够导入数据库的格式。被 posts 模块调用。
- (5) Visualize: 实现可视化图表生成，主要是生成词云和热力图。被 visual 模块调用。
- (6) MySQL: 实现后端与数据库的交互，包含数据库连接与数据插入、查询、删除、更新接口。被所有模块调用。

7.2.4 异常处理与安全逻辑

系统采用特定的逻辑处理异常，并保证一定的数据安全性。

异常处理逻辑方面，由后端接口统一捕获数据库访问和后端逻辑中可能出现的异常，返回 JSON 格式的错误信息。

安全逻辑方面，系统采用多种措施保障安全性：使用 bcrypt 对密码进行哈希存储，不在数据表中保存明文密码；使用基于 JWT 的身份认证机制，访问所有受保护接口均需携带 Access Token；当 Access Token 过期时，前端通过调用 /auth/refresh 获取新的令牌；文件上传接口限制文件后缀；等等。

访问部分 API 时，借助 token 信息二次检查当前登录用户的角色。用户角色分为 admin、analyst、ingest 三种，其中 admin 角色拥有管理系统用户的权限，analyst 角色拥有管理帖子数据的权限，包括导入、查询、删除、分析等，ingest 角色拥有查询帖子数据的权限，包括帖子条目查询、可视化查询和帖子数据导出。

7.3 具体功能编写

7.3.1 数据库建表与初始化

基于 5.1 的关系模型设计，使用以下 SQL 语句在数据库中建立数据表，并初始化该数据库应用系统的常用角色和管理员用户。

```
CREATE DATABASE social_sentiment;
use social_sentiment;

-- 用户表
CREATE TABLE users (
    user_id INT NOT NULL AUTO_INCREMENT,
    username VARCHAR(100) NOT NULL,
    platform VARCHAR(100) NOT NULL,
    gender VARCHAR(40) NOT NULL CHECK(gender IN ('Female', 'Male',
'Unknown')),
    birthday DATE,
    UNIQUE (username, platform),
    INDEX idx_username(username),
    PRIMARY KEY (user_id)
);

-- posts 表
CREATE TABLE posts (
    post_id INT NOT NULL AUTO_INCREMENT,
    user_id INT NOT NULL,
    content TEXT,
```

```

    location VARCHAR(255),
    create_time DATETIME,
    content_hash CHAR(32),
    INDEX idx_user(user_id),
    INDEX idx_create_time(create_time),
    INDEX idx_location(location),
    UNIQUE (user_id, content_hash, create_time),
    PRIMARY KEY (post_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- 情感分析表
CREATE TABLE sentiments (
    sentiment_id INT NOT NULL AUTO_INCREMENT,
    post_id INT NOT NULL,
    score DECIMAL(5, 4) NOT NULL,
    type VARCHAR(40) NOT NULL,
    model VARCHAR(40) NOT NULL,
    analysis_time DATETIME NOT NULL,
    INDEX idx_post(post_id),
    PRIMARY KEY (sentiment_id),
    UNIQUE (post_id, model),
    FOREIGN KEY (post_id) REFERENCES posts(post_id) ON DELETE CASCADE
);

-- 话题表
CREATE TABLE topics (
    topic_id INT NOT NULL AUTO_INCREMENT,
    topic VARCHAR(100) NOT NULL,
    description VARCHAR(255),
    INDEX idx_topic(topic),
    UNIQUE (topic),
    PRIMARY KEY (topic_id)
);

-- 关键词表
CREATE TABLE keywords (
    keyword_id INT NOT NULL AUTO_INCREMENT,
    keyword VARCHAR(100) NOT NULL,
    INDEX idx_keyword(keyword),
    UNIQUE (keyword),
    PRIMARY KEY (keyword_id)
);

```

```

-- 多对多关联表
CREATE TABLE posts_topics (
    post_id INT NOT NULL,
    topic_id INT NOT NULL,
    INDEX idx_topic(topic_id),
    PRIMARY KEY (post_id, topic_id),
    FOREIGN KEY (post_id) REFERENCES posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (topic_id) REFERENCES topics(topic_id) ON DELETE CASCADE
);

CREATE TABLE posts_keywords (
    post_id INT NOT NULL,
    keyword_id INT NOT NULL,
    INDEX idx_keyword(keyword_id),
    PRIMARY KEY (post_id, keyword_id),
    FOREIGN KEY (post_id) REFERENCES posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (keyword_id) REFERENCES keywords(keyword_id) ON DELETE
CASCADE
);

-- 应用系统用户认证/授权表（不同于 users 表中的推文作者）
CREATE TABLE sys_users (
    user_id INT NOT NULL AUTO_INCREMENT,
    username VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_active TINYINT(1) NOT NULL DEFAULT 1,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_username(username),
    PRIMARY KEY (user_id)
);

-- 角色表
CREATE TABLE sys_roles (
    role_id SMALLINT NOT NULL AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(255),
    PRIMARY KEY (role_id)
);

-- 用户-角色关系
CREATE TABLE sys_users_roles (
    user_id INT NOT NULL,
    role_id SMALLINT NOT NULL,
    PRIMARY KEY (user_id, role_id),

```



```

FOREIGN KEY (user_id) REFERENCES sys_users(user_id) ON DELETE CASCADE,
FOREIGN KEY (role_id) REFERENCES sys_roles(role_id) ON DELETE CASCADE
);

-- 初始化常用角色
INSERT INTO sys_roles (name, description) VALUES
    ('admin', '系统管理员'),
    ('analyst', '数据分析/管理权限'),
    ('ingest', '数据查询/导出服务');

-- 初始化管理员用户，密码为 admin123
INSERT INTO sys_users (username, password_hash) VALUES
    ('Administer',
'$2b$12$1dwwRYIzEqW/1D8HTzsszuSLj620dxDUzvu6irBTEF4oaY4sOGmni');
INSERT INTO sys_users_roles (user_id, role_id) VALUES
    (1, 1),
    (1, 2),
    (1, 3);

```

7.3.2 逻辑模块编写

后端按功能划分为 auth、posts、users、visual 四个 Blueprint，每个模块对外暴露 RESTful 风格 API。完整 API 参数、示例与完整实现代码已在源代码附件中呈现，此处只叙述关键设计和实现要点。

(1) auth 模块：提供核心路由 /auth/login、/auth/register、/auth/refresh、/auth/me、/auth/modify_password；使用 flask-jwt-extended 实现 token 生成、刷新和验证，使用 bcrypt 进行密码哈希和存储；前端统一使用 fetchWithAuth 函数访问需要 token 的路由，统一管理 access 和 refresh token。

(2) posts 模块：提供核心路由 /posts/import、/posts/export、/posts/query、/posts/delete、/posts/delete_sentiments、/posts/reanalyze。数据导入基于批量插入，查询、删除、情感分析均从接收前端的帖子信息后调用底层功能函数实现。进行角色校验，/posts/query、/posts/export 同时允许角色 ingest 和 analyst 访问，其余路由仅允许角色 analyst 访问。

(3) visual 模块：提供核心路由 /visual/sentiment_distribution、/visual/sentiment_trend、/visual/top_words、/visual/wordcloud、/visual/heatmap。情感分布饼状图、情感时间趋势折线图、词分布柱状图均在后端调用函数获得统计结果后由前端 Chart.js 绘制，词云由 wordcloud 生成 PNG 文件，热力图通过 geopy + 缓存获取地理编码、使用 folium 生成 HTML 文件。进行角色校验，所有可视化查询路由仅允许角色 ingest 访问。

(4) users 模块：提供核心路由 /users/query、/users/delete、/users/batch_update、/users/batch_set_role。各个管理操作相关路由，接收前端指定的用户后调用底层功能函数实现用户管理。进行角色校验，所有路由仅允许角色 admin 访问。

7.3.3 底层功能模块编写

底层功能模块包括 Password Encrypt、Sentiment Analysis、Extract Keywords、Data Normalizer、Visualize、MySQL，独立编写、为 Blueprint 提供核心逻辑服务。各个模块的完整代码已在源代码附件中呈现，此处同样只叙述关键设计和实现要点。

(1) Password Encrypt: 调用 bcrypt 库，实现密码加密与校验逻辑。

(2) Sentiment Analysis: 封装 TextBlob 与 BERT 两种情感分析方法，实现文本情感分类和打分，对外提供批处理接口。TextBlob 情感分析时，调用翻译 API 处理非英文文本；BERT 情感分析时，对单一文本分为多段进行分析，并基于一定规则转换情感分数、取各段的加权平均输出。

(3) Extract Keywords: 调用 sklearn 库中的 TfidfVectorizer 实现关键词提取，对外提供批处理接口。提取关键词时，对文本进行语言分析，并对中文和日文进行分词处理。

(4) Data Normalizer: 实现数据清洗与格式化，将不同平台采集的不同格式的数据转化成能够导入数据库的格式。源代码中只针对测试数据集格式编写了标准化函数 testdata_Normalizer()，后续可针对更多平台的数据格式编写标准化函数。

(5) Visualize: 实现可视化图表生成，其中生成词云在路由函数 /visual/wordcloud 中直接实现，生成热力图单独设计接口。

(6) MySQL: 实现后端与数据库的所有交互，分为插入、查询、删除、更新四个子模块实现接口，以及一个数据库连接函数。

插入子模块，主要实现插入帖子数据。批量插入时排除插入 users、posts、keywords、topics 表中已有的重复内容，采用“预查缓存 + INSERT IGNORE”策略，以降低重复查询与提高吞吐。

查询子模块，动态生成 SELECT 语句中的 WHERE 部分 + JOIN 部分 + 参数列表。该模块包含一个通过多种查询条件查询帖子 id 的函数，其他多数查询函数通过调用该函数、根据返回结果进行二次查询实现（包括帖子内容查询、情感分布和词分布查询等），实现了数据访问入口的统一、拥有较好的可复用性与可扩展性。此外，该模块还包含了用户和角色相关的查询。

删除子模块，对帖子和用户数据进行简单的 DELETE 操作。

更新子模块，包含更新帖子的分析记录、更新用户状态等功能函数，通过 INSERT、UPDATE、DELETE 等多种 SQL 语句实现。

7.4 功能测试

7.4.1 用户认证测试

测试目标：验证注册、登录、JWT 认证与权限控制逻辑，测试修改密码功能。

测试过程：

(1) 关键功能测试。

验证用户注册、登录、权限校验、修改密码的功能。

结果符合预期。用户能够成功注册和登录，并能够在修改密码后使用新密码登录成功。

图 7.2 用户认证测试：注册界面

(2) 进行异常处理能力测试，验证系统对异常情况的处理能力。

a. 注册时尝试留空用户名或密码、输入长度不足的密码等情况；登录时尝试输入错误的用户名或密码；修改密码时尝试输入长度不足的新密码、输入错误的旧密码等情况。系统阻止提交，显示错误提示。

b. 未登录状态下，请求任一需要 token 的受限资源(如帖子数据查询)。系统拒绝访问，显示错误提示。

测试结果：系统能正确执行用户注册、登录、修改密码的逻辑，同时生成并验证 Token、拒绝非法访问，功能正常。

7.4.2 用户管理测试

测试目标：验证管理员用户进行系统用户管理的逻辑，测试查询用户、删除用户、激活和冻结用户、重置用户密码、赋予和移除用户角色的功能。

测试过程：

(1) 关键功能测试。

预先注册数个系统用户，登录数据库初始化时创建的管理员用户（拥有角色“admin”），测试用户管理界面的用户列表显示、用户查询功能，以及对选中用户进行删除、冻结、激活、重置密码、赋予或移除角色的功能。

结果符合预期。用户管理窗口正确显示系统用户列表及各用户的用户名、id、角色、激活状态信息，且能够通过用户名查询指定用户；对选中用户的删除、冻结、激活、重置密码、赋予或移除角色操作均能够有效进行。



图 7.3 用户管理测试：用户查询展示



图 7.4 用户管理测试：用户列表展示

(2) 进行异常处理能力测试，验证系统对异常情况的处理能力。

a. 尝试在未选中用户的情况下执行删除 / 激活 / 冻结 / 重置密码 / 赋予角色 / 移除角色操作。系统不执行相关操作，显示错误提示。

b. 尝试对管理员用户执行删除 / 冻结操作。系统正确处理：当操作的用户对象包含非管理员用户时，该操作会跳过管理员用户；当操作的用户对象仅包含管理员用户时，系统不执行相关操作，显示错误提示。

测试结果：系统能正确执行测试查询、删除、激活、冻结用户的逻辑，以及重置密码、赋予和移除用户角色的功能，同时在对管理员用户进行操作时能够在一定程度上保护其权限，功能正常、兼顾安全性。

7.4.3 帖子数据管理测试

测试目标：验证帖子数据管理逻辑，包括导入、查询、删除帖子数据的功能，以及情感

分析、删除情感分析记录的功能，同时测试数据导入和情感分析的性能。

需要注意，由于通过 API 实时获取真实的社交媒体帖子数据成本较高，因此本课程设计中该系统仅包含历史数据导入功能，具体的实时数据抓取功能可以在实际应用中进行扩展。

测试数据导入功能时，使用 Kaggle 上的 COVID19 Tweets 数据集。该数据集包含了约 180,000 条来自平台 Twitter 的帖子，大多包含和 COVID19 有关的话题词或帖子推文内容，且带有发布位置、发布时间等信息，适合用于测试该系统的导入、查询、可视化功能。实际测试时，将该数据集划分为多个子数据集、分批导入测试，每个子数据集包含 5,000 条帖子。

测试过程：

(1) 关键功能测试。

登录拥有角色 “analyst” 的用户，打开数据管理窗口，测试帖子数据查询、数据导入功能，以及选中帖子进行删除、情感分析、删除情感分析记录的功能。

结果符合预期。窗口显示查询面板、数据导入面板和数据实时抓取面板（无具体功能，仅示意）；数据导入功能，能够正确导入 CSV 格式的数据集文件，并通过查询在数据列表上正确显示导入的帖子数据；对帖子进行删除、情感分析、删除情感分析记录的操作，均能够正确实现。

图 7.5 帖子数据管理测试：数据管理界面

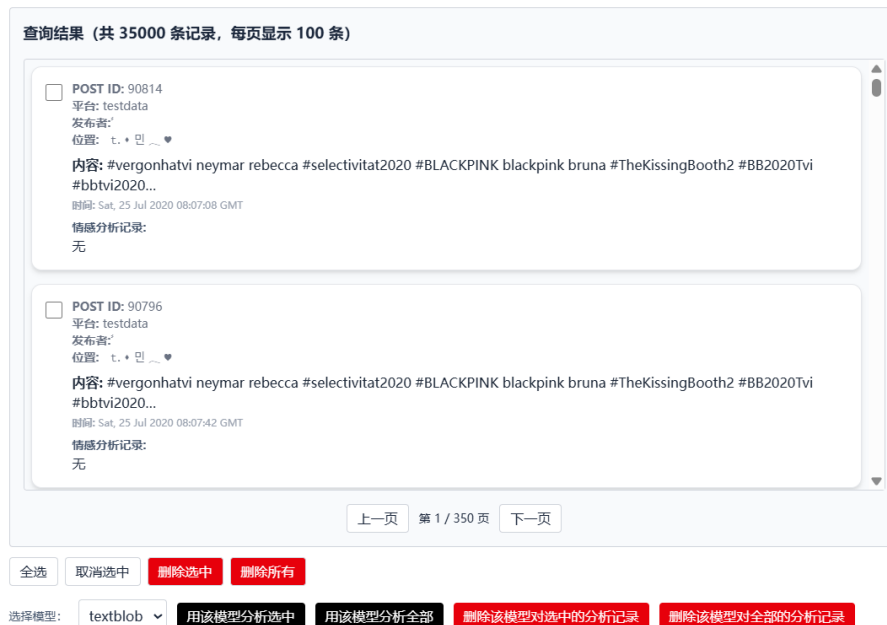


图 7.6 帖子数据管理测试：帖子列表展示

(2) 进行异常处理能力测试，验证系统对异常情况的处理能力。

- a. 尝试在未选择文件、选择错误文件格式、选择正确文件格式但是错误数据格式的情况下导入数据。系统不执行相关操作，显示错误提示。
- b. 尝试在未选中帖子的情况下执行删除、情感分析、删除情感分析记录操作。系统不执行相关操作，显示错误提示。

(3) 进行性能测试与分析。

a. 导入性能。

当数据库空表时，将 5,000 条标准化帖子导入（执行关键词提取，不执行情感分析），耗时约 31 s（约 161 条/s）；当数据库已有约 70,000 条记录时，导入同样 5,000 条标准化帖子，耗时约 34 s（约 147 条/s）。具体而言，已有约 70,000 条记录时，关键词提取过程用时约 5 秒，数据插入过程用时约 29 秒；更具体而言，数据插入的主要耗时过程为逐条插入 user、post、topics 和 keyword 数据，而通过查询来确认数据重复情况所占的时间相对较少。推测主要瓶颈为数据库写入以及批量插入时的事务成本。为提升吞吐，后续可采用多行插入策略，并在导入窗口临时调优 InnoDB 参数或先将部分索引延后建立。

b. 情感分析性能。

使用 TextBlob 对 5,000 条帖子进行情感分析，耗时约 45 s（约 114 条/s）。推测调用翻译 API 对 TextBlob 的分析效率影响较大，若限制帖子文本为英文则可删去翻译过程、可能达到更高的处理效率；另外，TextBlob 可能无法分析格式较特殊的帖子（具体现象为情感分数为 0），因此分析准确率存在波动、稳定性较差。可对于 TextBlob 的分析结果采用“情感分数为 0 则视为无效分析”的策略，以避免 TextBlob 的稳定性对统计结果造成干扰。

BERT 在 CPU 上对 5,000 条耗时约 480 s（约 10 条/s），普遍准确度高但代价大。后

续在硬件支持的基础上,可以将情感分析异步化并部署到专用 GPU 推理环境,或采用蒸馏、量化模型以在可接受精度下大幅加速。

测试结果:系统能够正确实现数据管理功能,且数据导入和情感分析的性能能够基本满足实时采集的需求。

未来的优化目标,可以从以下方面入手:短期内,使用多行插入和事务,加速数据导入;长期内,可将情感模型迁移到 GPU,并对前端进行虚拟化渲染与懒加载。

7.4.4 帖子查询与可视化测试

测试目标:验证帖子数据查询逻辑,包括基本帖子数据查询、数据导出、五种可视化查询功能。

以下测试中,示例的查询条件均为:话题“covid”,关键词“cases”,情感分析模型“bert”,其他条件留空,数据库中帖子总数约为 70,000 条。

测试过程:

(1) 关键功能测试。

登录拥有角色“ingest”的用户,打开数据查询窗口,测试帖子数据查询、数据导出、可视化查询功能。

结果符合预期。窗口显示数据查询面板和可视化查询面板;对于各类查询条件组合,帖子列表正确显示匹配查询条件的帖子信息,且能够导出 CSV 格式的帖子数据文件;能够根据当前查询条件,生成相应的“情感分布”“情感随时间变化趋势”“话题词热度”“关键词热度”“地点热力图”可视化查询结果。

数据查询与可视化

查询数据

时间范围 (左闭右开区间)

年 / 月 / 日

~

年 / 月 / 日

话题:

输入话题, 空格分隔

关键词:

输入关键词, 空格分隔

数据来源:

all

情感分析模型:

all

情感类别:

all

查询

导出 CSV

清空结果

可视化查询 (基于查询条件筛选)

情感分布 (饼状图)

注: 查询条件“情感类别”对该项无效

话题热度 (柱状图 + 词云)

话题词数量

20

关键词热度 (柱状图 + 词云)

关键词数量

20

情感随时间变化趋势 (折线图)

地点热力图 (地图)

图 7.7 帖子查询与可视化测试: 查询界面

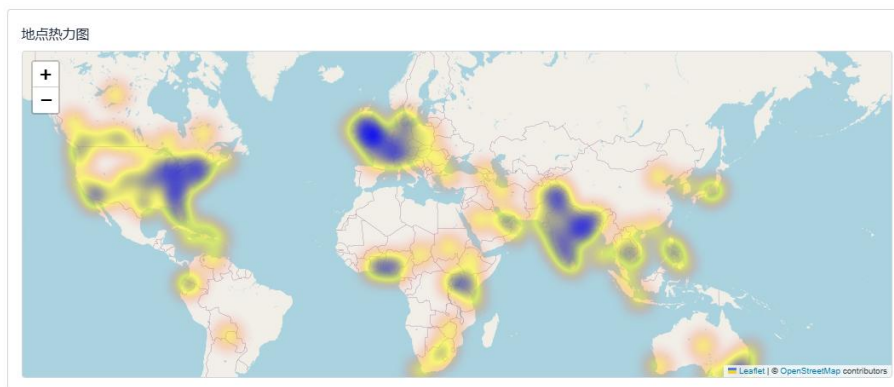


图 7.9 帖子查询与可视化测试：可视化查询结果

(2) 进行异常处理能力测试，验证系统对异常情况的处理能力。

- a. 尝试在查询结果为空的情况下导出数据。系统不执行相关操作，显示错误提示。
- b. 尝试在查询结果为空的情况下生成各类可视化查询结果图。系统不执行相关操作，显示错误提示。

(3) 进行性能测试与分析。

a. 数据查询性能。

当数据库已有约 35,000 条记录时测试查询功能，发现响应延迟主要由前端渲染占用。帖子数据查询界面，帖子列表每页显示 50 条数据、仅作为数据展示，点击“查询”到结果渲染完成的延迟小于 2 秒；帖子数据管理界面，帖子列表每页显示 1,000 条数据、兼顾“全选”按钮的便利性，当查询结果不少于 1,000 条数据时，点击“查询”到结果渲染完成的延迟大约需要 4 秒。

未来可以减少每页展示条目、使用虚拟滚动或懒加载，显著降低用户感知延迟。

b. 可视化查询性能

设定相应查询条件，测试 5 种可视化查询的响应时间。对于情感分布、情感随时间变化趋势、话题 / 关键词热度查询，系统均能在较短时间内（延迟小于 5 秒）显示统计图形；对于情感热力图，由于所调用的 geopy 库的地理位置转经纬度 API 存在访问限制，因此生成速度较慢，对于 1866 条数据的热力图，生成用时约 630 秒，在可应用的范围之外。

程序中已经使用缓存来避免重复查询、加速经纬度提取过程；未来可以进一步改用访问限制较少的 API 或离线地理库来提取经纬度，加速情感热力图的生成。

测试结果：系统正确实现了帖子数据查询、数据导出、各类可视化查询功能，且大多数数据查询功能的性能能够满足用户需求，少数（热力图生成功能）需要进一步优化。

八. 总结

本次课程设计围绕“社交媒体活动的时空分布与情感倾向监测”，从需求分析、可行性分析、概念设计、逻辑设计到系统实现、测试与性能评估，进行了完整的数据库应用系统搭建的流程实践。系统实现部分采用 Python + Flask 作为后端、MySQL 作为数据库管理系统，前端使用原生 HTML + JavaScript 并以 Tailwind CSS 快速构建界面；在情感分析上同时支持轻量模型 TextBlob 与深度模型 BERT，并实现关键词提取、时序/话题统计及饼图、折线、词云、热力图等可视化输出。开发过程中，基于 E-R 模型完成了关系模型的设计，保证数据库满足第三范式（3NF），选取了针对性索引和若干联合唯一约束以提升查询性能，并在插入逻辑中通过批量查询与缓存降低重复查库，从而在数万级别记录条件下仍能保持可接受的吞吐与响应。

总体而言，项目在课程设计范围内实现了预期的功能目标，数据模型、接口设计与前后端协作方案清晰、可复现；在性能上能够满足中小规模数据分析和演示需求，但若面向更大规模或生产级应用，应重点改进批量写入策略、情感分析的并行与硬件加速策略，以及地理位置解析的鲁棒化处理。后续工作方向包括：将情感分析与关键词提取异步化并外置推理服务、在导入窗口采用边写边索引或后置索引构建以提升吞吐、引入分区和分库设计以便于水平扩展。该系统已具备扩展为更高吞吐与实时性的基础框架，适合在未来接入更丰富的数据源和更高性能的分析基础设施以服务实际舆情监控场景。

参考文献

- [1] Pallets. Welcome to Flask — Flask Documentation (3.1.x)[EB/OL]. (2025-08-19)[2025-10-21]. <https://flask.palletsprojects.com/>
- [2] Oracle. MySQL :: MySQL Documentation[EB/OL]. (2025-01-01)[2025-10-21]. <https://dev.mysql.com/doc/>
- [3] TextBlob. TextBlob: Simplified Text Processing[EB/OL]. (2025-01-13)[2025-10-21]. <https://textblob.readthedocs.io/>
- [4] Andreas Mueller. WordCloud for Python documentation[EB/OL]. (2024-11-10)[2025-10-21]. https://amueller.github.io/word_cloud/
- [5] Rob Story. Folium [EB/OL]. (2025-01-01)[2025-10-21]. <https://python-visualization.github.io/folium/>
- [6] GeoPy Contributors. Welcome to GeoPy's documentation![EB/OL]. (2023-11-24)[2025-10-21]. <https://geopy.readthedocs.io>
- [7] Tailwind Labs Inc. Tailwind CSS - Rapidly build modern websites without ever leaving your HTML.[EB/OL]. (2025-01-01)[2025-10-21]. <https://tailwindcss.com/>
- [8] Chart.js. Chart.js[EB/OL]. (2025-10-13)[2025-10-21]. <https://www.chartjs.org/>
- [9] scikit-learn developers. scikit-learn: machine learning in Python[EB/OL]. (2025-09-01)[2025-10-21]. <https://scikit-learn.org/>
- [10] Hugging Face. Hugging Face – The AI community building the future.[EB/OL]. (2025-10-20)[2025-10-21]. <https://huggingface.co/>
- [11] Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[EB/OL]. (2018-10-11)[2025-10-21]. <https://arxiv.org/abs/1810.04805>